

INFORME ESTRUCTURA INTERNA DEL SOFTWARE DE FLIGHTGEAR

1. Introducción

FlightGear es un simulador de vuelo de código abierto diseñado bajo una arquitectura modular que permite separar claramente los distintos subsistemas implicados en la simulación. Esta característica lo convierte en una herramienta especialmente adecuada tanto para entornos académicos como para aplicaciones de investigación y desarrollo.

El objetivo de este documento es describir en detalle la organización interna del simulador, explicando el propósito de cada módulo, el flujo de información entre ellos y la forma en que se define el comportamiento de una aeronave mediante los motores de simulación YASim y JSBSim. Para ello, se analizan además dos casos prácticos: la Tecnam P92 (YASim) y la Cessna 172P (JSBSim).

2. Análisis de la arquitectura general de FlightGear

FlightGear se estructura como un conjunto de subsistemas desacoplados que cooperan entre sí mediante una estructura común de datos. Esta arquitectura permite que cada módulo tenga una responsabilidad bien definida, facilitando la extensibilidad del sistema.

2.1 Núcleo del sistema (FGCore)

El núcleo del simulador es el encargado de coordinar la ejecución de todos los subsistemas. Su función principal es gestionar el ciclo de simulación, asegurando que cada módulo se ejecute en el orden adecuado y que la información fluya correctamente entre ellos.

Este núcleo no implementa directamente la lógica de simulación, sino que actúa como un **orquestador**, sincronizando la entrada de datos, el cálculo físico y la salida gráfica.

El flujo de datos que procesa este sistema es:

- **Entrada:** Usuario interactúa con UI, hardware o scripts.
- **Actualización de Property Tree:** Los comandos modifican valores de control (timón, alerones, acelerador).
- **Simulación:** Motor FDM (YASim o JSBSim) lee el estado actual, calcula fuerzas y movimientos, y actualiza el Property Tree.
- **Visualización y paneles:**
 - OSG lee las variables de la Property Tree y renderiza la escena 3D.
 - Instrumentos y HUD actualizan sus displays.

- **Output:** Información visible, sonido, telemetría, y logs.

Este conjunto se ejecuta de forma continua, lo que permite la sincronización entre todos los sistemas.

2.2 Property Tree: Núcleo de comunicación del sistema

El Property Tree constituye el elemento central de la arquitectura de FlightGear. Se trata de una estructura jerárquica que almacena todas las variables del simulador, organizadas en forma de árbol.

Su función principal es permitir la comunicación entre subsistemas sin que estos dependan directamente unos de otros. En lugar de intercambiar información de forma directa, todos los módulos leen y escriben en el Property Tree.

- Por ejemplo:
- El usuario actúa sobre el joystick → se actualiza `/controls/flight/aileron`
- El modelo de vuelo lee esa variable → calcula la dinámica
- El resultado se escribe en `/orientation/roll`
- El sistema gráfico utiliza ese valor para representar el movimiento
- Este mecanismo convierte al Property Tree en una capa de abstracción fundamental, garantizando coherencia y sincronización en todo el sistema.

2.3 Nasal (Scripting Engine)

El lenguaje Nasal permite extender el comportamiento del simulador mediante scripts.

A través de Nasal se pueden implementar:

- Lógica de sistemas de la aeronave
- Automatización de procesos
- Instrumentación personalizada

Al igual que el resto de los módulos, Nasal interactúa directamente con el Property Tree.

2.4 Interfaz de Usuario (UI)

Gestiona menús, ventanas, HUD, paneles 2D/3D, y enlaces con el Property Tree para actualizar valores en tiempo real. FlightGear ofrece varios mecanismos de interacción con la UI:

- **Comandos y bindings:** Teclas, joysticks y periféricos controlan variables del Property Tree mediante mapeo directo.

- **Paneles y HUD:** Representaciones gráficas que se actualizan continuamente desde el Property Tree.
- **Sockets y red:** Permite recibir datos externos o enviar telemetría a aplicaciones externas (ej. integración con Arduino o telemetría en tiempo real).
- **Telemetría y logging:** exportación a CSV o transmisión vía red (FGCom, sockets).

Todas estas entradas se traducen en modificaciones sobre el Property Tree, lo que permite que el sistema de simulación procese la información de forma uniforme.

2.5 Visualización

FlightGear usa OpenSceneGraph (OSG) como motor gráfico para representar el entorno de simulación.

El sistema de visualización no calcula física, sino que interpreta los valores del Property Tree para:

- Posicionar la aeronave en el espacio
- Animar superficies de control
- Renderizar la cabina e instrumentos
- Mostrar el entorno 3D

Esto asegura que la representación visual sea coherente con el estado físico calculado por el modelo de vuelo.

2.6 Motor de simulación (FDM – Flight Dynamics Model)

Es el encargado de calcular el comportamiento físico de la aeronave (fuerzas, momentos, posiciones y velocidades de cada ciclo de simulación).

- Módulos principales:
 - **YASim:** aerodinámica basada en geometría de componentes.
 - **JSBSim:** ecuaciones de vuelo 6DOF con coeficientes de aerodinámica.
- Es independiente de la visualización y la UI, solo interactúa con el Property Tree.

De este modo, la arquitectura de FlightGear puede entenderse como un sistema en el que todos los subsistemas se comunican exclusivamente a través del Property Tree. Cada módulo cumple una función específica (entrada, cálculo, representación o lógica), pero ninguno depende directamente de los demás, lo que permite un alto grado de desacoplamiento y flexibilidad en el diseño del simulador.

3. Flujo de ejecución del simulador

El comportamiento global del simulador puede entenderse como una cadena de transformación de información en el que participan todos los subsistemas, donde los datos fluyen desde la entrada del usuario hasta la representación visual pasando por el modelo dinámico de vuelo.

El flujo es el siguiente:

1. **Entrada de usuario**
El usuario introduce comandos mediante dispositivos físicos o sistemas externos.
2. **Actualización del Property Tree**
Estas entradas modifican variables de control como alerones, timón o potencia.
3. **Cálculo del modelo dinámico (FDM)**
El motor de simulación (YASim o JSBSim) lee el estado actual y calcula fuerzas, momentos y aceleraciones.
4. **Actualización del estado del avión**
Los resultados se escriben en el Property Tree (posición, velocidad, orientación).
5. **Visualización**
El sistema gráfico renderiza la escena en función de estos valores.
6. **Instrumentación y UI**
Los paneles muestran información actualizada al usuario.

Este ciclo se repite continuamente, permitiendo una simulación en tiempo real coherente.

3.1 Ejemplo de flujo completo de información en FlightGear

Para comprender completamente la interacción entre los distintos módulos del simulador, se procede a analizar un caso práctico de funcionamiento.

Supongamos que el usuario actúa sobre el joystick para inclinar el avión hacia la derecha mediante el control de alerones.

En primer lugar, la entrada del usuario es capturada por el sistema de entrada y traducida a una modificación en el Property Tree, concretamente en la variable `/controls/flight/aileron`.

A continuación, el motor de simulación (FDM) accede a esta variable y la utiliza como entrada para el cálculo de la dinámica de vuelo. En este proceso, se determinan las fuerzas aerodinámicas generadas por la deflexión de los alerones, lo que produce un momento de alabeo.

Como resultado de este cálculo, el FDM actualiza variables como /orientation/roll y /velocities/angular, que describen el nuevo estado dinámico de la aeronave.

Posteriormente, el sistema de visualización (OpenSceneGraph) lee estos valores y actualiza la representación gráfica del avión, mostrando el movimiento de alabeo en tiempo real.

De forma paralela, los instrumentos de cabina y paneles virtuales acceden al Property Tree para mostrar la nueva información al usuario.

Este flujo evidencia que el modelo de simulación actúa como núcleo de cálculo, mientras que el resto de subsistemas se encargan de proporcionar entradas o interpretar los resultados, manteniendo siempre el Property Tree como punto central de intercambio de información, permitiendo una simulación coherente y sincronizada.

4. Modelo dinámico de vuelo del simulador

El modelo dinámico de vuelo es el responsable de calcular el comportamiento físico de la aeronave.

El motor de simulación no interactúa directamente con la interfaz de usuario ni con el sistema gráfico. Esto implica que el modelo dinámico puede ser sustituido o modificado sin necesidad de alterar el resto de subsistemas, siempre que mantenga la misma interfaz de comunicación basada en el Property Tree.

Su única interfaz de comunicación es el Property Tree, del cual obtiene las entradas necesarias y al que devuelve los resultados del cálculo. Este desacoplamiento permite sustituir el modelo dinámico sin afectar al resto del sistema.

Una característica fundamental de FlightGear es que este modelo no está embebido en el código del simulador, sino definido mediante archivos externos, lo que permite modificar completamente el comportamiento sin recompilar el software.

FlightGear soporta varios motores de dinámica de vuelo, pero los más empleados son:

4.1 YASim (Yet Another Simulation)

Modelo de simulación basado en componentes aerodinámicos y geometría del avión.

- **Estructura:**

- Cada superficie del avión se modela como un componente (ala, fuselaje, cola, hélices).
- Calcula fuerzas y momentos aerodinámicos a partir de la velocidad relativa del aire, ángulo de ataque y parámetros geométricos.
- Integración explícita de ecuaciones de movimiento (6DOF).
- **Ventajas:** Fácil de configurar a partir de geometría, adecuado para aviones ligeros o prototipos, rápido prototipado.
- **Limitaciones:** Menos preciso en condiciones extremas, simplificación aerodinámica respecto a CFD en comparación con otros modelos basados en datos experimentales.

En este enfoque:

- La aeronave se descompone en superficies (alas, cola, fuselaje)
- Las fuerzas aerodinámicas se calculan a partir de dicha geometría
- No se utilizan coeficientes aerodinámicos explícitos

Esto permite definir el comportamiento del avión de forma relativamente sencilla, siendo adecuado para prototipos o aeronaves ligeras.

4.2 JSBSim

Motor de simulación más avanzado basado en ecuaciones de vuelo (riguroso, basado en aerodinámica teórica y experimental).

- **Estructura:**
 - Input: archivos XML que definen aerodinámica, motores, tren de aterrizaje y sistemas del avión.
 - Calcula dinámicas 6DOF mediante integración numérica de ecuaciones diferenciales.
 - Modela sistemas más complejos (combustible, sistema hidráulico, autopilot).
- **Ventajas:** Alta precisión, adecuado para simulaciones académicas y experimentales, gran flexibilidad y adaptación.
- **Limitaciones:** Configuración más compleja, requiere datos aerodinámicos precisos.

En este caso:

- Se utilizan coeficientes aerodinámicos tabulados
- Se modelan sistemas complejos (motor, combustible, control)
- Se integran ecuaciones diferenciales en 6 grados de libertad

Este enfoque permite una mayor precisión, siendo adecuado para simulación avanzada y entornos de investigación.

4.3 Comparativa YASim vs JSBSim

Aspecto	YASim	JSBSim
Base de cálculo	Geometría + aerodinámica simplificada	Ecuaciones 6DOF + coeficientes aerodinámicos precisos
Complejidad de configuración	Baja	Alta
Precisión física	Media (buena para prototipos)	Alta (apto para investigación)
Sistemas modelados	Limitados (hélices, flaps)	Completos (motor, hidráulico, eléctrico)
Flexibilidad	Media	Alta
Integración en FG	Nativa y simple	Nativa, requiere parsing XML completo

5. Estructura de aeronaves en FlightGear

Las aeronaves en FlightGear se organizan como conjuntos de archivos estructurados en directorios y cada carpeta cumple una función específica:

Aircraft/<modelo>/

- Models/ geometría 3D y animaciones
- Systems/ definición del modelo de vuelo
- Engines/ motores y hélices del avión
- Nasal/ scripts de lógica de sistemas
- Sounds/ sonidos característicos del avión
- Dialogs/
- *.xml configuración general

Esta estructura permite separar claramente la física, la visualización y la lógica del sistema.

Cada modelo de simulación se organiza de forma diferente:

- **YASim:**
 - Archivo aircraft-yasim.xml define masas, superficies, propulsión, geometría.
 - Archivos .xml de geometría y perfiles de ala (wing.xml, propeller.xml).

- Parámetros aerodinámicos derivados de geometría (coeficientes de lift/drag simplificados).
- **JSBSim:**
 - Archivos XML con definición completa: aircraft.xml, control.xml, propulsion.xml.
 - Cada elemento aerodinámico se define con coeficientes experimentales o tabulados.
 - Puede integrarse con controladores automáticos (autopilot, PID externo, sistemas de vuelo).

6. Ejemplo YASim (Tecnam P92)

En el modelo Tecnam P92, el comportamiento de la aeronave está completamente definido en un único archivo YASim (p92W-yasim.xml).

Este archivo incluye:

- Geometría del avión
- Masa y centro de gravedad
- Superficies aerodinámicas
- Sistema de propulsión

Concentra el modelo de vuelo completo, de manera que cambios en ala, centro de gravedad o propulsión alteran directamente la respuesta dinámica de la aeronave.

Esto implica que cualquier modificación del comportamiento del avión se realiza directamente sobre este archivo, modificando parámetros geométricos.

Por ejemplo:

- Sustentación → modificando el ala
- Estabilidad → modificando el centro de gravedad
- Rendimiento → modificando motor y hélice

Este enfoque simplifica el desarrollo, pero limita la precisión del modelo.

6.1 Estructura

Presenta una estructura bastante simplificada:

- Tecnam-P92/
 - Models/
 - p92.ac

- p92.xml
- Systems/FDM/
 - p92A-yasim.xml
 - p92W-yasim.xml
- Nasal/
- *-set.xml

7. Ejemplo JSBSim (Cessna 172p)

En el caso de la Cessna 172P, el modelo es más complejo y modular.

El archivo principal (c172p.xml) define:

- Aerodinámica
- Masas e inercias
- Superficies de control
- Interacción con motor y hélice

Sin embargo, el comportamiento completo del avión depende también de otros archivos:

- Motor → eng_io320.xml
- Hélice → prop_*.xml
- Sistemas → carpeta Systems

Esto permite modificar cada subsistema de forma independiente, aumentando la flexibilidad y precisión.

7.1 Estructura

El modelo JSBSim presenta una estructura más extendida:

- c172p/
 - c172p.xml, FDM (JSBSim) “núcleo del FDM”
 - c172p-set.xml, Configuración inicial
 - c172p-main.xml, Integración de aeronave con FlightGear
 - Engines/
 - eng_io320.xml, Planta motriz
 - eng_io360.xml
 - prop_*.xml, Hélices
 - Systems/, Sistemas

- Models/, Modelo 3D
- Nasal/, Lógica y scripting
- *-set.xml, Variantes del avión

8. Conclusión

FlightGear presenta una arquitectura altamente modular basada en el uso del Property Tree como núcleo de comunicación.

El sistema permite desacoplar completamente:

- Entrada de usuario
- Simulación física
- Visualización
- Lógica de sistemas

El análisis de los modelos de aeronave muestra dos enfoques claramente diferenciados:

- YASim, orientado a simplicidad y rapidez.
 - El modelo Tecnam P92 basado en YASim sigue una arquitectura centralizada en un único archivo FDM (p92W-yasim.xml), basado en la geometría del avión y orientado a la simplicidad y prototipado de aeronaves, lo cual implica que la modificación sea más sencilla a costa de una menor fidelidad física en la simulación.
 -
- JSBSim, orientado a precisión y flexibilidad.
 - El modelo C172P basado en JSBSim sigue una arquitectura centralizada en un archivo FDM (c172p.xml) pero extendida mediante distintos módulos según las dependencias del avión (motor, hélices, sistemas, etc). Esto permite que se consiga una alta fidelidad física a la hora de simular un vuelo, además de una gran capacidad de modificación e integración con sistemas más complejos.